P1482R0
Modules Feedback

Published Proposal, 2019-02-08

This version:
http://wg21.link/P1482R0

Authors:
Bruno Cardoso Lopes (Apple)
Michael Spencer (Apple)
JF Bastien (Apple)

Audience:
EWG, SG15

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P1482R0.bs

Table of Contents

Informative References

As the C++ committee closes in on C++20, several concerns have emerged regarding the state of C++ Modules and whether they should be promoted from a TS to the C++20 draft. Legitimate questions are being asked, and while it's unclear what the future has in store for C++20, we consider the current merging proposal in [P1103R1] to have achieved a nice tradeoff between the different perspectives on how a modular system for C++ should work.

That doesn't mean that the current proposal solves all problems in the C++ universe or shouldn't be improved any further; there's immense room for development and a set of open questions. For good (or bad) we lay out some of the hot topics, as seen in post-San Diego and pre-Kona mailings, and our current take on some of these issues:

- C++ Modules are the C++ Savior
- Module preamble, configuration and dependency scanning
- Deploying modules & scalability
- Standard Library

This paper isn't advocating for or against modules being merged into the C++20 draft. It's a mere overview of publicly available facts we think are relevant to the current discussion.

§ 1. C++ Modules are the C++ Savior

They're not. It would be great if Modules could solve all problems in the C++ universe, but that's unrealistic. It's probably a good opportunity to make the world a better place (e.g. moving away from macros), but you can't get the benefits of componentization without introducing well-defined dependencies.

We consider that C++ Modules must solve componentization and provide build time speedups. However, packaging or a binary distribution system sound out of scope. At the same time, C++ Modules shouldn't prevent any of those ideas from flourishing independently. We think that the current model meets both requirements, as explained in more detail in this paper.

The addition of layers and abstractions won't ever be free. We ask the reader to keep that in mind while balancing the tradeoffs.

§ 2. Module preamble, configuration, and dependency scanning

The module preamble consists of a set of module import declarations (module imports and header imports) and is terminated by the first non import decl. Module imports never import macros, while macros from header imports are visible directly after the closing '`;`'. The macros that are imported from headers can influence subsequent import logic. Includes can appear within the preamble as long as they only expand into import decls.

In C++17, dependency scanning requires a correct preprocessor. The current state of modules is no different. It is no slower to determine the dependencies of a modular C++ file than a normal C++ file today, and current proposals aimed at restricting the preamble wouldn't make a scanner any faster.

After scanning the preamble, there may be an `#include` later in the file, but since this is uncommon a scanner can do a quick scan for `#`*white-space*`include` and fall back to preprocessing the entire file if required.

§ 2.1. Module configuration

An important topic is whether to ship module artifacts, but it's not clear whether that's actually feasible because of how modules are configured at build time.

For example, implementations allow configuration macros via `-D`, which is useful for specifying dynamic library import/export (within in a build) or platform-specific constants (when a library needs special handling for each platform). Each variation in configuration macros needs a different module build artifact.

Another potential issue in shipping module artifacts consistently handling command-line flags controlling diagnostics. Suppose you build module `A` with `-Wno-error`. Next, someone tries to invoke the compiler with `-Werror` while re-using module `A`. What should happen here? Should the compiler accept that? Should the user give up because their workflow isn't supported? This can get tricky for delayed diagnostics in template instantiations, where the declaration is in the module and the instantiation is in a user of the module.

A mechanism like the artifact hashing in Clang Modules is an example of how to solve this problem. When building a module with Clang Modules, the hash for the artifact location consists of: command line macros, language options, and a set of similar compiler flags. Changes in any of those by a subsequent compiler invocation typically triggers a new module build.

None of the above prevents fine-grained memoization. Indeed, the current Clang approach can evolve to consider configurations at a finer granularity. Our experience so far makes it unclear whether this

use case is worth addressing.

§ 2.2. Dependency scanning can be fast

The LLVM project is investing in a prototype which provides dependency scanning for headers with 4–10× speed up over `-Eonly` preprocessing. There's a clear path to compute C++ Modules dependencies using this approach. We are therefore confident that tooling can be done fast and reliably.

If you think about Clang Modules as a legacy header only C++ modules, the approach should be straightforward to relate. This is how it works:

- Each header file has its source code minimized, by trimming down anything that is not `#define`, `#include`, `#import`, `@import`, see the code for a detailed explanation.
- An `#include` triggers a lookup to find out if a header is part of a module. If it is, every header that maps to the same module is bundled and serialized into a `.pcm` (precompiled module). The mapping is done using external files called module maps. Each `#include` of a header in a previously serialized bundle with the same configuration is essentially free.
- While traversing the mapping above, installed listeners register necessary dependencies.

This can be further optimized by additional levels of caching. Since system modules (such as system headers) do not change frequently, it might be feasible to pre-populate metadata with dependency info, or even module artifacts, for a chosen set of common configurations.

§ 2.3. Mapping modules

Another topic related to dependency scanning and build times is how the mapping between module names and files should happen. As stated in [P1427R0], [P0778R0], and other discussions, this deserves more investigation. However, it doesn't need to block C++ Modules. There are interesting side effects of treating this as an implementation detail:

- Modern compilers will have significant freedom to explore solutions for out of process services, in-memory (or virtual) filesystems, and distributed builds; i.e., models other than a traditional filesystem.
- Existing module systems will have more flexibility in transitioning to a C++ modules world.
- Compiler extensions that enable bridging to other languages—such as Objective-C and WebAssembly—also have more flexibility.

- Darwinism at its finest: let the best solution become de facto practice, which the Committee can explore in a separate Standing Document.

§ 2.4. Modules and Package Management

We consider source code to be *the* C++ shipping format. Modules aren't the right place to solve longstanding vendor compatibility problems. Neither are Modules the right vehicle for distribution of C++ packages.

This derives from the consistency problems mentioned in the modules configuration section above. It's unrealistic to consider that one could ship a module and at the same time make it consumable by a different users with distinct restrictions and setups.

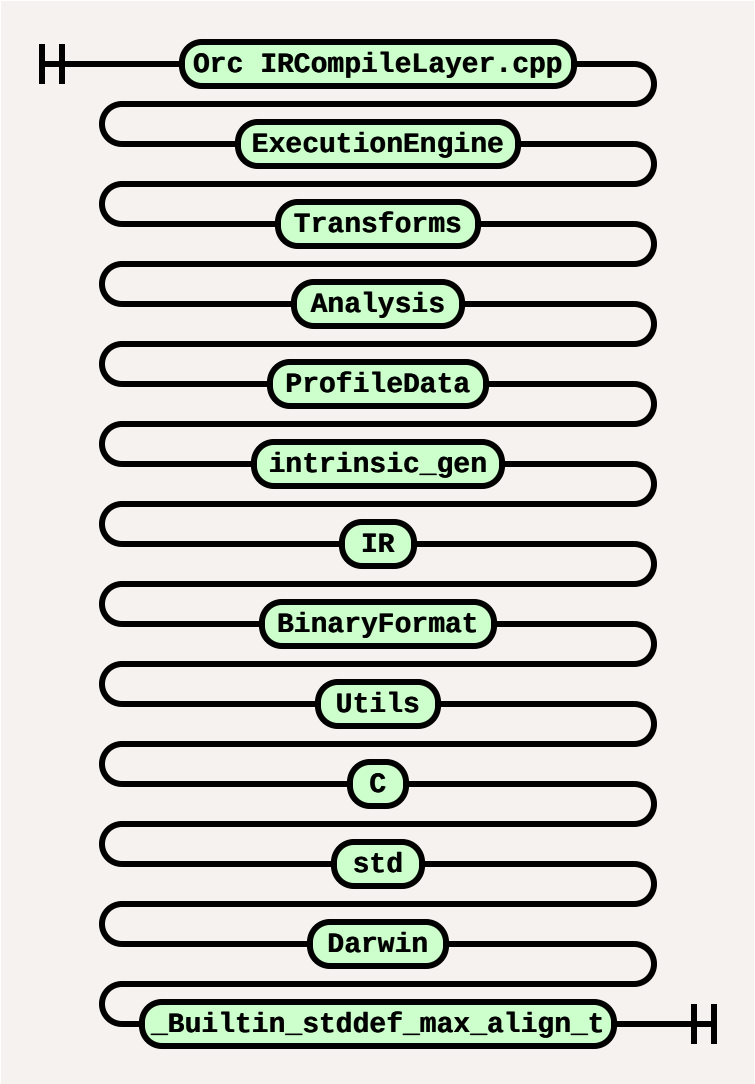
We believe modules should be built (not shipped or distributed) as part of a build, and potentially shared in the same environment for other compiler invocations that end up using non-conflicting setups.

However, the nice thing about not specifying any of this at the standards level is that vendors are free to explore with both models, or even come up with a mixed mode, where modules could be shipped for a specific audience. It would then be on that vendor to figure out how to support that binary format over time, and how to keep their implementation fast while making interoperability easy. That said, we don't think that speculations about how Modules impact packaging should block the progress of C++ Modules.

§ 3. Deploying Modules: does it scale?

Let's take LLVM as an example of how a modular code base might behave. LLVM is currently modularized with Clang Modules. These are similar to a version of C++ Modules where each module only provides a *module interface unit*, as in GCC's BMIs. Since the BMIs don't depend on module implementation units, this is a fair approximation of a C++ Modules dependency subgraph. Here is the Clang Modules build graph for LLVM:

- 2440 `.cpp` files which *import* modules. None of those are part of a module themselves, since there's no similar concept of a *module implementation unit*.
- 173 module files (`.pcm`) including all different configurations for a given module. For instance, `LLVM_Utils` shows up 5 times, covering 5 different configurations.
- 14330 *import* edges modeling the total import relationships between `.cpp` to `.pcm`, and `.pcm` to `.pcm`.
- The longest import path has 12 hops, illustrated below.



Most of the speed benefit from using these modules come from re-using the artifacts for these 5 configurations across 2440 C++ TUs. In a clean build of LLVM using Clang Modules, we see about a 30% speedup (`10m10s` vs `13m44s`) using `ninja -j6`. A more complete survey including different configurations should cover more accurate performance results, but this illustrates one data point commonly found at a developer's desk.

It's possible to build a strawperson example which performs poorly, but this is an example of a real C++ project that benefits from the reuse of some form of module artifacts.

This example uses Clang's implicit modules in which each compiler invocation lazily builds the modules it depends on, and often time multiple compiler invocations are building the same module. Building modules lazily doesn't solve the problem of the added build dependencies. You still have to build the same number of modules before you can proceed with the translation unit, only now multiple TUs are doing duplicate work instead of sharing it. There's also no real parallelism exposed because the preamble must be the first thing and no real parsing work will be done until the dependencies have been compiled.

§ 4. Standard Library

Yet another contentious topic is Standard Library modularization. [P1427R0] expresses a desire that the Standard Library be modularized along with the standardization of modules. [P1453R0] brings interesting points, such as the rare opportunity for reorganization and the limited time for C++20. Our current understanding is that standard library modularization shouldn't be a requirement for the standardization of modules.

In the clang Modules world, there's modularization support for libc++. Supporting this modularization was not straightforward given the potential non-modular nature of headers and the underlying includes from other system headers. We end up with a module map that has a lot of workarounds and does not fully respect layering. Although C++ Modules are different from clang Modules, some of the problems here are the same: to do it right, a rethink of organization and layering is a *must*. We don't trust that any rush to get this in will be beneficial at this point. The same might not be true for other implementations of the standard library, but there's a chance other vendors might also be abusing header inclusion and preprocessor tricks.

The types of issues we describe for the standard library are the types of issues regular developer code will encounter when modularizing their code base.

§ References

§ Informative References

[P0778R0]
Nathan Sidwell. Module Names. 10 October 2017. URL: https://wg21.link/p0778r0

[P1103R1]
Richard Smith. Merging Modules. 8 October 2018. URL: https://wg21.link/p1103r1

[P1427R0]
Peter Bindels, Ben Craig, Steve Downey, Rene Rivera, Tom Honermann, Corentin Jabot, Stephen Kelly. Concerns about module toolability. 20 November 2018. URL: https://wg21.link/p1427r0

[P1453R0]
Bryce Adelstein Lelbach. Modularizing the Standard Library is a Reorganization Opportunity. 21 January 2019. URL: https://wg21.link/p1453r0